Problem 1

Consider the maze shown on Fig. 4. This maze consists of several walls that the agent cannot enter, and *bumps* and *oils* such that moving to them yields negative rewards. For simplicity, consider an 18×18 matrix, where each element is associated with one of the following cell types:

- *Empty*
- *Wall*
- *Bump*
- *Oil*

State Space ($\mathcal{S}$): The state space contains all cells in the maze except the walls, where the agent can possibly be ($18 \times 18 - 76$ (walls) $= 248$).

Action Space ($\mathcal{A}$): The agent can take one of the four possible actions at any given state: *up (U), down (D), right (R), and left (L)*.

Transition Probabilities: After taking any action, with probability $1 - p$, the agent moves to the anticipated next state (the cell in the chosen direction). With probability $p/2$, the agent moves to each of the two perpendicular neighboring states (each with probability $p/2$). The agent never moves in the opposite direction. If the selected next state is a wall, the agent stays in the current state.

Consider the following examples.

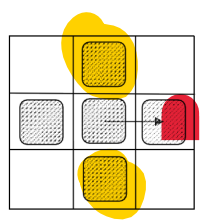
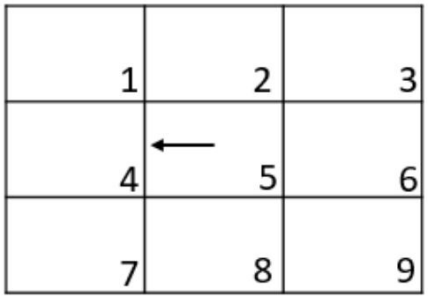


Figure 1: Stochastic transition for action R.

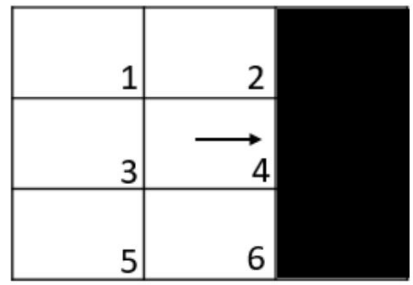
If the action *right (R)* is selected, the agent

$$\begin{cases} \text{moves to the state in the direction of action with probability } 1-p, \\ \text{moves to each perpendicular neighboring state with probability } p/2. \end{cases}$$

Notice that if the selected next cell is a wall, the agent stays in the current cell.



(a) Action *Left (L)* from $s = 5$.



(b) Action R from $s = 4$ (next state wall).

$$
\begin{aligned}
&p(s' = 4 \mid s = 5,\ a = L) = 1 - p,\\
&p(s' = 2 \mid s = 5,\ a = L) = p/2,\\
&p(s' = 8 \mid s = 5,\ a = L) = p/2,\\
&p(s' = 5 \mid s = 5,\ a = L) = 0,\\
&p(s' = 6 \mid s = 5,\ a = L) = 0.
\end{aligned}
$$

$$
\begin{aligned}
&p(s' = 4 \mid s = 4,\ a = R) = 1 - p,\\
&p(s' = 2 \mid s = 4,\ a = R) = p/2,\\
&p(s' = 6 \mid s = 4,\ a = R) = p/2,\\
&p(s' = 3 \mid s = 4,\ a = R) = 0.
\end{aligned}
$$

Figure 2: Transition-probability examples for the stochastic motion model.

Reward Function: The primary objective is to find the optimal policy that leads the agent to the goal state with the maximum accumulated reward. Therefore, we define:

- reward of taking any action: -1 (delay),
- reward for ending up in an oil state: -5,
- reward for hitting a wall: -0.8,
- reward for ending up in a bump state: -10,
- reward for reaching the goal state: $+200$.

For instance, this can be expressed for the example below as:

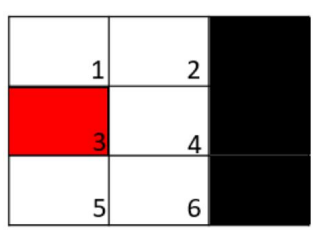


Figure 3: Local reward illustration (oil cell in red and wall boundary)

$$
\begin{aligned}
&R(4, L, 3) = -6 && (-5 \text{ for oil and } -1 \text{ for taking an action})\\
&R(4, L, 2) = -1 && (\text{for taking an action})\\
&R(4, L, 6) = -1 && (\text{for taking an action})\\
&R(4, R, 4) = -1.8 && (-1 \text{ for taking an action and } -0.8 \text{ for hitting a wall}).
\end{aligned}
$$

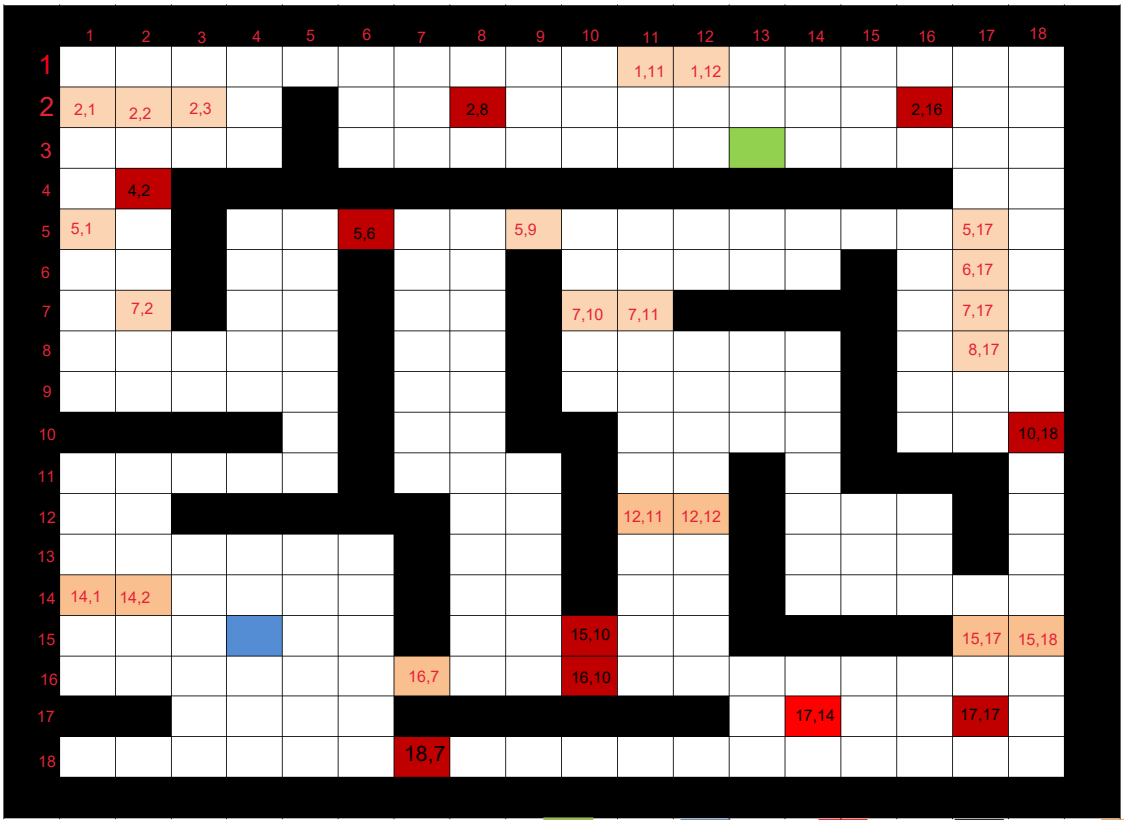


Figure 4: Maze for Problem 1 (18×18).

Bump Wall Oil Start End

1) Policy Iteration (vector form)

Use the vector-form Policy Iteration technique to find the optimal policy and the optimal state values. Choose action *Left* in all states as your initial policy for Policy Iteration. Solve the problem for the following cases:

a. $p = 0.02,\ \gamma = 0.99,\ \theta = 0.01$ (Base Scenario)

b. $p = 0.4,\ \gamma = 0.99,\ \theta = 0.01$ (Large Stochasticity Scenario)

c. $p = 0.02,\ \gamma = 0.4,\ \theta = 0.01$ (Small Discount Factor Scenario)

Report the optimal value functions, the optimal policy, and the optimal path from start to end. Meanwhile, compare the results obtained in the three scenarios (see the example figures later in this problem statement).

2) Value Iteration (vector form)

Repeat the same process as part (1) with the vector-form Value Iteration method. Set the initial state values to zero. Compare the results obtained from Value Iteration and Policy Iteration.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   - $\Delta \leftarrow 0$
   - Loop for each $s \in \mathcal{S}$:
     - $v \leftarrow V(s)$
     - $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \big[r + \gamma V(s')\big]$
     - $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   - *old-action* $\leftarrow \pi(s)$
   - $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
   - If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Figure 5: Reference pseudo-code: Policy Iteration (vector/matrix form).

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma V(s')\big]$

Figure 6: Reference pseudo-code: Value Iteration (vector/matrix form).

Example of required results

For each combination of *method* and *scenario*, you need to provide the following set of plots.

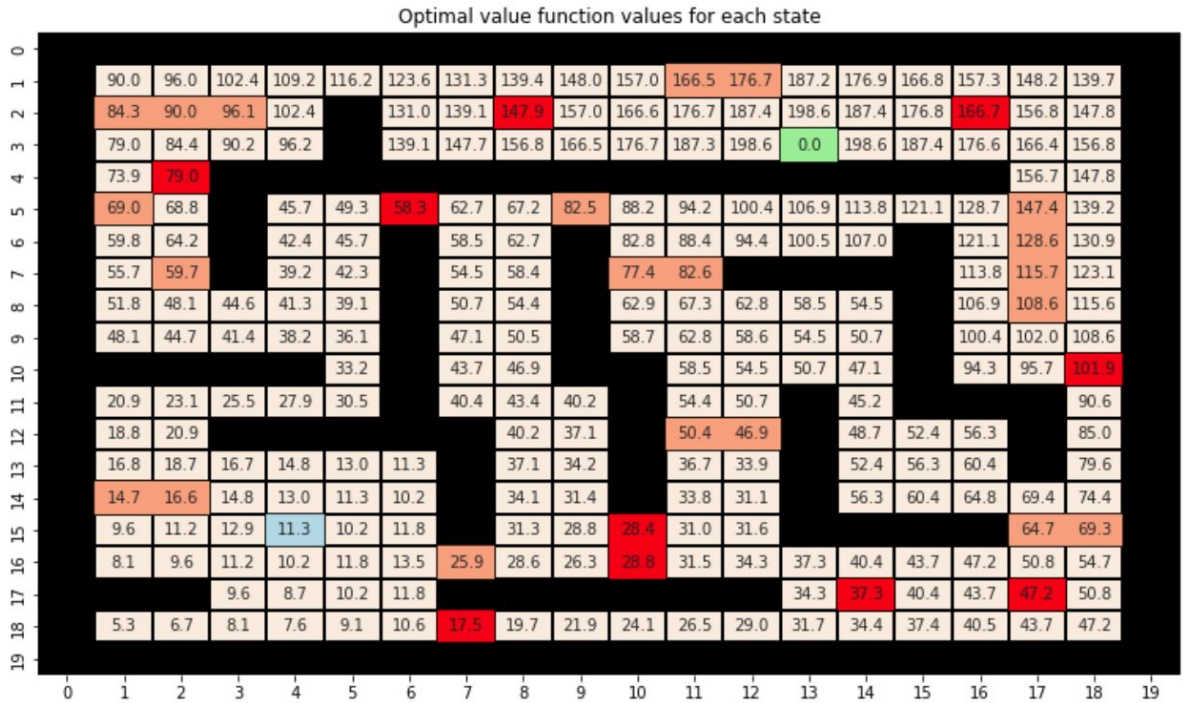


Figure 7: Example: optimal value function at all states for the base scenario under Policy Iteration.

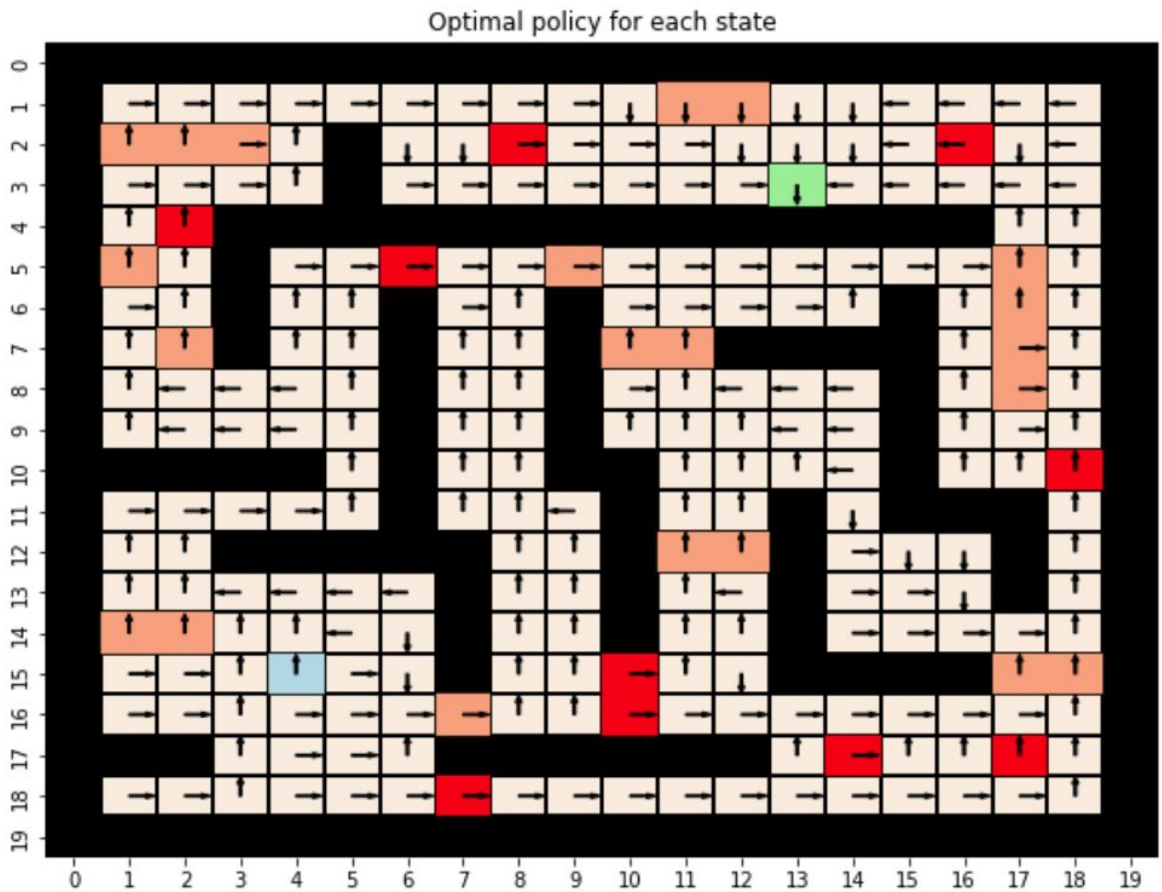


Figure 8: Example: optimal policy at all states for the base scenario under Policy Iteration.

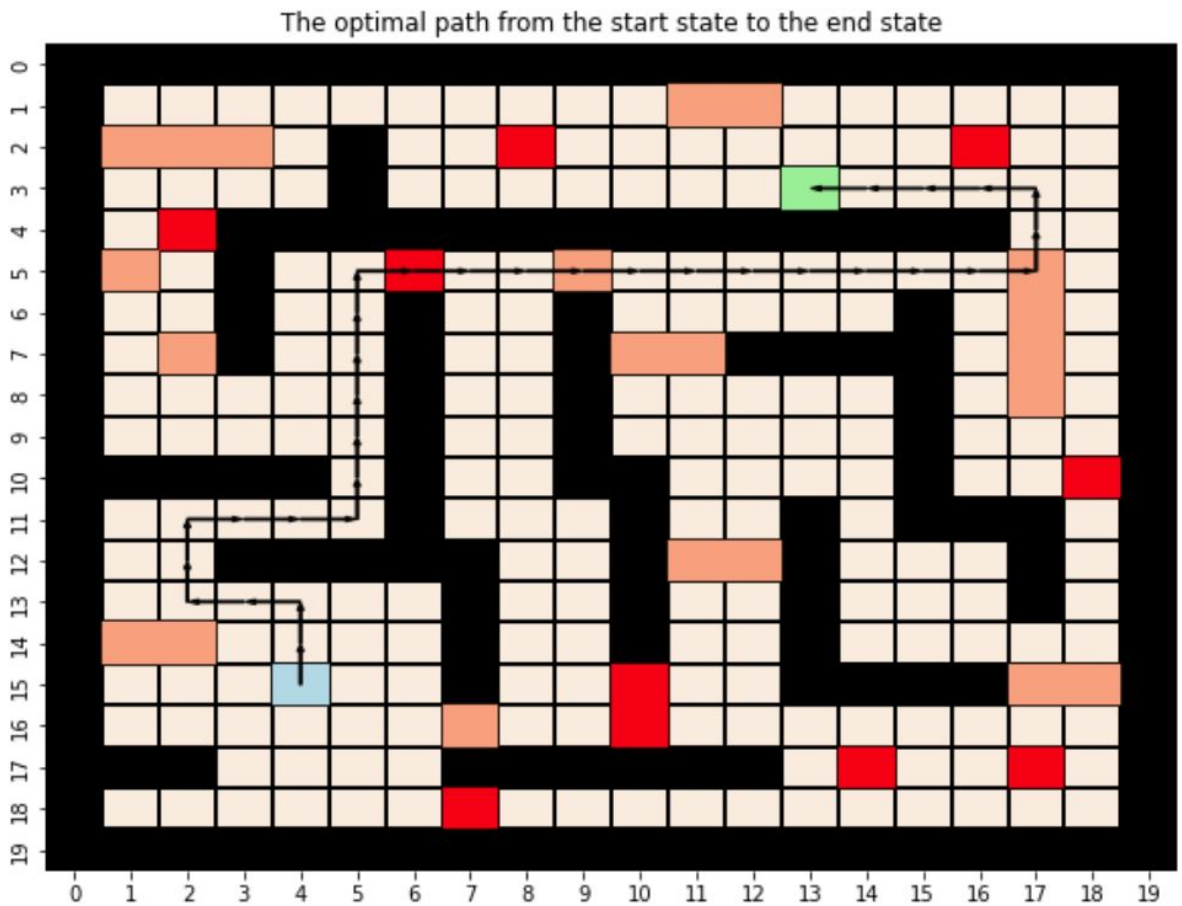


Figure 9: Example: optimal path from the start state to the end state for the base scenario under Policy Iteration.

An episode starts at the given start state and terminates upon reaching the goal state. To avoid infinite loops under stochastic transitions, we also impose a maximum horizon of $T_{\max}$ steps; if the goal is not reached by $T_{\max}$, the episode terminates at $T_{\max}$. In this project, use $T_{\max} = 400$ steps for all simulations and trajectory-based plots.

When plotting a "path" for a given policy in the stochastic maze, report a single sampled trajectory obtained by executing the policy and sampling state transitions according to the transition probabilities (with the same start and goal). Because transitions are stochastic, repeated executions can yield different paths even under the same policy.

You also need to compare the results across all three scenarios, explain the differences, and share your thoughts on the results. Please also report:

- the number of iterations of Policy Iteration (number of times that policy improvement has been performed), and
- the number of iterations of Value Iteration.

3) Effect of stochasticity.

Using the setting $p = 0.02,\ \gamma = 0.99,\ \theta = 0.01$ and either Value Iteration or Policy Iteration (your choice), repeat the solution for

$$p \in \{0.02,\ 0.2,\ 0.6\}, \qquad \gamma = 0.99,\ \theta = 0.01,$$

while keeping the *same* start and goal states.

a. **Path overlay.** For each $p \in \{0.02, 0.2, 0.6\}$, generate *two* independent sampled start-to-goal trajectories by executing the optimal policy and sampling transitions according to the stochastic motion model. Plot these two trajectories on the same maze figure for that p (i.e., one figure per p), using distinct markers/line styles and a legend. Briefly comment on how the variability between the two trajectories changes as p increases.

b. **Average cumulative reward curves.** For each $p \in \{0.02, 0.2, 0.6\}$, simulate 10 trajectories from the same start state until reaching the goal or until the maximum horizon $T_{\max}$. Let $\tau^{(i)}$ denote the termination time of trajectory i (goal reached or truncation at $T_{\max}$), and define the plotting horizon for that p as

$$T_p = \max_{i \in \{1,\ldots,10\}} \tau^{(i)} \le T_{\max}.$$

Define the cumulative reward of trajectory i up to step t as $G^{(i)}(t) = \sum_{k=0}^{t} r_k^{(i)}$. If a trajectory terminates at time $\tau^{(i)} < T_p$, keep its cumulative reward constant afterward, i.e.,

$$G^{(i)}(t) = G^{(i)}(\tau^{(i)}) \quad \text{for all } t \ge \tau^{(i)}.$$

Then, for $t = 0, 1, \ldots, T_p$, compute and plot on the same figure the average cumulative reward curve

$$\bar{G}(t) = \frac{1}{10} \sum_{i=1}^{10} G^{(i)}(t).$$

Use a legend for the three values of p and briefly comment on the observed trends.

4) Effect of bump penalty.

Using the setting $p = 0.02,\ \gamma = 0.99,\ \theta = 0.01$, change the bump penalty from -10 to -50 (all other rewards unchanged).

- Provide the optimal policy plot and the optimal path plot for the modified bump penalty scenario.
- Compare with the original bump penalty case (-10) and discuss whether the policy/path becomes more conservative around hazards.

Problem 2

The left part of Figure 10 shows the pathway diagram of a well-known biological network (the p53-Mdm2 negative feedback loop). The network consists of four components (genes), represented in the state vector.

$$\mathbf{s}_k = [\text{ATM},\ \text{p53},\ \text{WIP1},\ \text{MDM2}]^T.$$

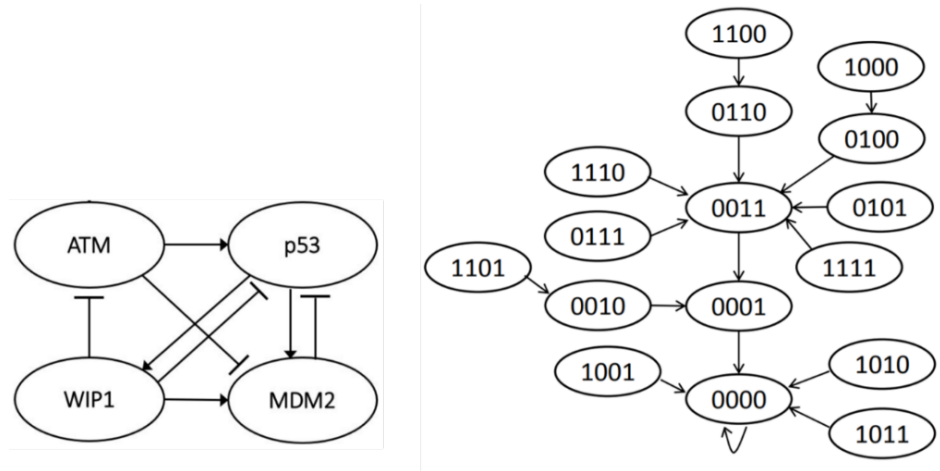


Figure 10: Pathway diagram for the p53-MDM2 network (left), and state transition diagram (right).

At any given time, the state value of each gene can be one of two values: 0 (OFF) and 1 (ON). Thus, the state space consists of $2^4 = 16$ states, for example:

$$\mathbf{s}^1 = [0,0,0,0]^T,\ \mathbf{s}^2 = [0,0,0,1]^T,\ \mathbf{s}^3 = [0,0,1,0]^T,\ \mathbf{s}^4 = [0,0,1,1]^T,\ \ldots,\ \mathbf{s}^{16} = [1,1,1,1]^T.$$

The action space is 每个基因取 0 或 1，所以一共 2^4=16 个状态

$$\mathcal{A} = \left\{\mathbf{a}^1 = [0,0,0,0]^T, \mathbf{a}^2 = [1,0,0,0]^T, \mathbf{a}^3 = [0,1,0,0]^T, \mathbf{a}^4 = [0,0,1,0]^T, \mathbf{a}^5 = [0,0,0,1]^T\right\},$$

meaning that a single action at any time should be selected from $\mathcal{A}$, i.e., $\mathbf{a}_{k-1} \in \mathcal{A}$.

Dynamics and transition matrices

The dynamics (state transition probabilities) of this system are governed by

$$\mathbf{s}_k = \overline{C\,\mathbf{s}_{k-1}} \oplus \mathbf{a}_{k-1} \oplus \mathbf{n}_k, \tag{1}$$

where C is the connectivity matrix

$$C = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 1 & 0 & -1 & -1 \\ 0 & 1 & 0 & 0 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$\overline{\mathbf{v}}$ maps each element of a vector to 1 if it is greater than 0 and to 0 otherwise, $\mathbf{n}_k$ is the state transition noise, and "$\oplus$" is component-wise modulo-2 addition (XOR), for example

$$\begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

XOR（异或）规则很简单，两个数相同取0，不同取1

We assume that the four elements of $\mathbf{n}_k$ are i.i.d. Bernoulli with parameter p, i.e., for $j \in \{1, 2, 3, 4\}$,

$$\mathbf{n}_k(j) \sim \begin{cases} 1 & \text{with prob. } p, \\ 0 & \text{with prob. } 1-p. \end{cases}$$

The controlled transition matrices for this system are 16×16 matrices. The element in the ith row and jth column of the transition matrix associated with action $\mathbf{a}$ is

$$\big(M(\mathbf{a})\big)_{ij} = p^{\left\|\mathbf{s}^j - \left(\overline{C\mathbf{s}^i} \oplus \mathbf{a}\right)\right\|_1} (1-p)^{4 - \left\|\mathbf{s}^j - \left(\overline{C\mathbf{s}^i} \oplus \mathbf{a}\right)\right\|_1}, \tag{2}$$

where $\|\mathbf{v}\|_1 = \sum_\ell |v(\ell)|$.

For the system under no control, $\mathbf{a} = [0, 0, 0, 0]^T$, the (noise-free) state transition diagram is shown on the right of Figure 10. Without process noise, the system reaches state "0000" after a few transitions, spending most of its time there.

Objective and reward

The objective is to control (perturb) the system so that the state values of all components are 1 (ON) at all time steps (a continuous task). In other words, the system is desired to have as many active components as possible, ideally spending most of its time in the state "1111". This is expressed through the reward function

$$R(\mathbf{s}, \mathbf{a}, \mathbf{s}') = 5\,\mathbf{s}'(1) + 5\,\mathbf{s}'(2) + 5\,\mathbf{s}'(3) + 5\,\mathbf{s}'(4) - \|\mathbf{a}\|_1 . \tag{3}$$

According to this reward function, the activation of each gene has a reward of $+5$, and actions $\mathbf{a}^2$ to $\mathbf{a}^5$ have a reward of -1.

Dynamic programming

After constructing the controlled transition matrices, use matrix-form Dynamic Programming with $\gamma = 0.99$.

a) For $p = 0.045$, use matrix-form Value Iteration to compute the optimal control policy π^*, which is a vector of size 16 specifying the best action at any given state. Use all-zero initial state values and $\theta = 0.01$. Compare the average activation of genes, $AvgA$ (definition is provided below), under the obtained optimal policy with $AvgA$ under a no-control policy (taking $\mathbf{a}^1$ in all states).

b) Repeat part (a) for $p = 0.18$ and $p = 0.55$. What is your observation? Compare the obtained optimal policy and $AvgA$ for these two noise parameters with the results of part (a).

c) Similar to part (a), use $p = 0.045$, and action $\mathbf{a}^1$ in all states as the initial policy. Perform matrix-form Policy Iteration to compute the optimal control policy π^*. Compare the results with part (a). Are the obtained policies the same?

Please also report:

- the number of iterations of Policy Iteration (number of policy improvement steps), and
- the number of iterations of Value Iteration.

Policy evaluation by simulation

In addition to the final value function and policy (similar to Problem 1), evaluate the performance of the final obtained policy during testing as follows.

In episode i, start with a random initial state $\mathbf{s}_0 \in \{\mathbf{s}^1, \ldots, \mathbf{s}^{16}\}$. Perform the action prescribed by the optimal policy, $\mathbf{a}_0 = \pi^*(\mathbf{s}_0)$, and use the transition matrix associated with $\mathbf{a}_0$ to sample $\mathbf{s}_1 \sim p(\mathbf{s}' \mid \mathbf{s}_0, \pi^*(\mathbf{s}_0))$. Continue this process to obtain a length-150 trajectory

$$D^i = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{a}_{149}, \mathbf{s}_{150}), \quad \text{where } \mathbf{a}_k = \pi^*(\mathbf{s}_k).$$

The average activation rate in episode i is

$$A^i = \frac{1}{150} \sum_{k=1}^{150} \|\mathbf{s}_k\|_1 , \qquad 0 \le A^i \le 4, \tag{4}$$

where values closer to 4 correspond to near-perfect activation of all four genes at all time steps, and values close to 1 denote that a single gene (among the four) has been activated on average.

Repeating the same process for 75 episodes, the average activation rate is computed as

$$AvgA = \frac{1}{75} \sum_{i=1}^{75} A^i. \tag{5}$$

Questions: Email TA(Amirhossein) at ravari.a@northeastern.edu.

Appendix: Reference notes (as provided in the handout)

Value Iteration – Matrix Form

Transition matrices $M(a)$, $a \in A$, Reward $R^a_{ss'}$, Number of states N.

Hadamard Product

- Reward: $R^a_s = \left(M(a) \odot R^a_{ss'}\right) 1_{N\times 1}$, for $a \in A$.
- $V_0 = 0_{N\times 1}$, $k=0$

Repeat

- Value Iteration Backup: $V_{k+1} = \max_{a\in A} \left\{ R^a_s + \gamma M(a) V_k \right\}$ (Row-wise)
- $k = k+1$

Until $\max_{i\in\{1,\dots,N\}} |V_k(i) - V_{k-1}(i)| < \theta$

- Optimal state values: $V^* = V_k$
- optimal policy: $\pi^* = \arg\max_{a\in A} R^a_s + \gamma M(a) V^*$ (Row-wise)

Figure 11: Matrix-form Value Iteration reference note.

Policy Iteration: Matrix Form

Initialization: arbitrary policy π, transition matrices $M(a)$, Reward $R^a_{ss'}$

Hadamard Product

- Reward: $R^a_s = \left(M(a) \odot R^a_{ss'}\right) 1_{N\times 1}$, for $a \in A$.

Policy Evaluation

- $M(\pi) = \begin{bmatrix} \text{Row 1 of } M(\pi(s')) \\ \vdots \\ \text{Row N of } M(\pi(s^N)) \end{bmatrix}$, $R^\pi_s = \begin{bmatrix} \text{Element 1 of } R^\pi_s(s') \\ \vdots \\ \text{Element N of } R^\pi_s(s^N) \end{bmatrix}$

- $V^\pi = \left(I - \gamma M(\pi)\right)^{-1} R^\pi_s$

Policy Improvement

Row-wise

- $\pi' = \arg\max_{a \in A} R^a_s + \gamma M(a) V^\pi$

If $\pi \neq \pi'$, $\pi = \pi'$ and go back to policy Evaluation step.

Else

- optimal policy: $\pi^* = \pi'$
- optimal state values: $V^* = V^{\pi}$

Figure 12: Matrix-form Policy Iteration reference note.